Experimental Analysis of Flow Table Saturation and Dynamic Path Redirection in OpenFlow Networks

Environment: Mininet 2.3.0 | Open vSwitch 2.17.9 | Ubuntu Linux | **Status:** Validated & Fully Reproducible

1 Executive Summary & Research Scope

Software-Defined Networking (SDN) decouples control logic from physical data forwarding. This investigation evaluates an Open vSwitch (OVS) network under two conditions: (1) flow table saturation via synthetic rule-bloat and cache-bypassing microflows, and (2) automated, latency-triggered dynamic rerouting to mitigate bottleneck link contention. Experiments were conducted under uniform link constraints: bandwidth $B = 10.0\,\text{Mbps}$, delay $d = 5.0\,\text{ms}$, and queue limit $q = 100\,\text{packets}$.

2 Network Topology & Forwarding Matrix

The topology comprises four hosts (h_1–h_4) across six OpenFlow 1.3 switches (s_1–s_6).

Path	Endpoints & Role	Forwarding Sequence	Hop Count / Theoretical Min RTT
p_1	$H1 \leftrightarrow H4$ [Primary]	$h_1 \leftrightarrow s_1 \leftrightarrow s_2 \leftrightarrow s_3 \leftrightarrow s_4 \leftrightarrow h_4$	5 links (4 switches) $\rightarrow$ 50.0 ms
p_2	$H2 \leftrightarrow H3$ [Cross-Traffic]	$h_2 \leftrightarrow s_2 \leftrightarrow s_3 \leftrightarrow h_3$	3 links (2 switches) $\rightarrow$ 30.0 ms
p_3	$H1 \leftrightarrow H4$ [Bypass]	$h_1 \leftrightarrow s_1 \leftrightarrow s_5 \leftrightarrow s_6 \leftrightarrow s_4 \leftrightarrow h_4$	5 links (4 switches) $\rightarrow$ 50.0 ms

3 Phase 1: Baseline Bottleneck Latency (Plot 1)

Synthetic UDP flows (λ_{14}) were injected from 1.0 to 9.6 Mbps over Path p_1. The minimum propagation delay is 5 links $\times$ 5.0 ms $\times$ 2 = 50.0 ms.

Injected Rate λ_{14} (Mbps)	Mean RTT T (ms)	Buffer Queue	Observed Behavioral Classification
1.0 – 7.0	50.30 – 50.57	0 packets	Unsaturated propagation regime
8.0 – 8.5	50.28 – 50.30	≤ 2 packets	Linear operational boundary
9.0	50.33	5–8 packets	Incipient queue formation
9.3	60.43	~ 22 packets	Knee region: non-linear queuing growth
9.6	150.30	~ 95 packets	Severe buffer saturation ($q = 100$)

4 Phase 2: Flow Table Saturation Attack (Plot 2)

Ingress switch s_1 was populated with 3,000+ wildcard/priority OpenFlow entries. Dynamic probes varying destination Layer-4 ports (1000–4000) bypassed OVS kernel caching (EMC), forcing userspace classifier evaluations and producing a clear lookup delay offset.

Rate λ_{14} (Mbps)	Plot 1 RTT (ms)	Plot 2 RTT (ms)	Overhead ΔT	Dominant Delay Factor
1.0	50.57	56.92	+6.35 ms	Userspace classifier search overhead
3.0	50.47	56.61	+6.14 ms	Userspace classifier search overhead
5.0	50.41	56.71	+6.30 ms	Userspace classifier search overhead
8.0	50.30	56.78	+6.48 ms	Userspace classifier search overhead
9.3	60.43	67.57	+7.14 ms	Combined lookup delay and queuing
9.6	150.30	157.07	+6.77 ms	Bottleneck link buffer saturation

5 Phase 3: Dynamic Congestion Rerouting (Plot 3)

With nominal traffic $\lambda_{14}^{*} = 5.0\,\text{Mbps}$ active on Path p_1, cross-traffic $\lambda_{23} = 7.0\,\text{Mbps}$ was initiated on link s_2–s_3 at $t = 8.7\,\text{s}$ (total load: 12.0 Mbps $> B$). The queue filled, driving latency to 166.0 ms. At $t = 9.8\,\text{s}$, crossing the 2$\times$ threshold (102.64 ms) triggered `route_p3.sh` via the controller, restoring latency to 51.4 ms over backup path p_3.

Timeline (s)	Measured RTT	Network Operating State	Active Path ($H_1 \leftrightarrow H_4$)
$t = 0.0 - 7.6$	$50.9 - 51.8$ ms	Undisturbed baseline; zero cross-traffic	Path p_1 (s_1-s_2-s_3-s_4)
$t = 8.7$	51.2 ms	Cross-traffic begins on s_2–s_3 (7.0 Mbps)	Path p_1 (queue filling begins)
$t = 9.8$	166.0 ms	Threshold crossed; `route_p3.sh` executed	Rerouting from $p_1 \rightarrow p_3$
$t = 11.5 - 32.1$	$50.6 - 51.9$ ms	Latency recovered; congestion isolated to p_2	Path p_3 (s_1-s_5-s_6-s_4)

6 Complete Experimental Reproduction Runbook

```
# 1. Environment Preparation
sudo pkill -9 -f iperf

# 2. Execute Plot 1 Baseline Benchmark
# Terminal 1:
sudo python3 project_topo.py

# Terminal 2:
sudo bash route_p1.sh
python3 bench_baseline_latency.py
python3 bench_baseline_latency.py --reverse

# 3. Inject Attack Rules and Execute Plot 2 Benchmark
python3 inject_cache_pollution.py -n 4000
python3 bench_table_saturation.py

# 4. Flush Attack Rules and Execute Plot 3 Dynamic Rerouting
sudo bash route_p1.sh
python3 bench_dynamic_reroute_plot3.py
```